

Capítulo 14

Objetos y Clases

Hasta ahora en el curso hemos visto dos paradigmas de programación: un paradigma funcional, en donde los problemas se modelan como funciones que toman datos de entrada y retornan un valor simple o compuesto que sólo depende de la entrada, y un paradigma imperativo, en donde hay estructuras que actúan como memoria y por lo tanto los resultados retornados por las funciones no sólo dependen de los valores de los parámetros de la función, sino que también dependen del estado actual de estas estructuras. A partir de este capítulo se estudiará un tercer paradigma de programación, conocido como *programación orientada al objeto*. En este paradigma de programación se utilizan dos conceptos fundamentales: objetos y clases. Éstos forman la base de toda la programación en lenguajes orientados a objetos.

Un *objeto* es un modelo computacional de un ente o concepto que posee ciertos atributos y con el cual podemos realizar ciertas operaciones. Hasta el momento, hemos visto datos complejos (*structs*) que nos permiten modelar los atributos del concepto, y definimos funciones sobre éstos para implementar las distintas operaciones posibles. En cambio, en la programación orientada a objetos, los objetos contienen tanto los datos asociados a éstos como las operaciones que se pueden realizar con ellos. Una *clase* permite describir en forma abstracta los atributos y operaciones del concepto modelado, que luego se instancia en un objeto. En este capítulo estudiaremos como crear objetos y como interactuar con ellos, dejando para el próximo capítulo cómo se define una clase en Python.

14.1 Un ejemplo: automóviles

Suponga que necesita escribir un programa que permita hacer una simulación sobre tráfico de automóviles. Por cada automóvil de la simulación, es necesario almacenar información como: color, velocidad actual, velocidad máxima, cantidad de gasolina en el estanque, etc. Además, se requiere poder realizar las siguientes operaciones con los automóviles: acelerar, frenar, apagar, consultar cuánta gasolina queda en el estanque, cambiar su color, etc. En programación orientada a objetos, para resolver este problema es necesario definir una clase `Automovil` de la cual podamos crear objetos (instancias específicas de automóviles), y luego a través de interacciones con estos objetos programar la simulación.

14.2 Crear e interactuar con objetos

Como ya se mencionó, antes de ver cómo podemos crear nuestras propias clases vamos a ver cómo crear objetos. Supongamos que ya tenemos implementada la clase `Automovil`. Vamos a ver en un pequeño ejemplo cómo *instanciar* distintos objetos.

```
1 >>> unAutomovil = Automovil()
```

Con este pequeño código lo que hemos hecho ha sido crear un objeto de la clase `Automovil`.

Nuestro automóvil también puede moverse, acelerar, frenar, etc. Los objetos tienen *comportamiento*, siguiendo nuestro modelo de la realidad. Supongamos que nuestros objetos de la clase `Automovil` tienen distintas funciones, o *métodos*, que realizan acciones sobre el objeto:

```
1 >>> unAutomovil.encender()
2 >>> unAutomovil.acelerar()
3 >>> unAutomovil.frenar()
4 >>> unAutomovil.apagar()
```

Nuestros métodos, al ser funciones, también pueden recibir parámetros. Un método indica qué parámetros recibe (en su contrato). Por ejemplo, nuestro automóvil puede esperar saber cuánto tiempo presionar el acelerador:

```
1 unAutomovil.acelerar(30) # presiona el acelerador durante 30 segundos
```

Estas llamadas a métodos del objeto son aplicaciones de funciones como las hemos visto siempre, salvo que su contexto está unido al *estado* de cada objeto.

En el ejemplo anterior, supongamos que nuestro automóvil entrega el nivel de gasolina *actual* tras un tiempo en movimiento:

```
1 >>> gasolina = unAutomovil.obtenerNivelGasolina()
2 >>> if gasolina > 0:
3     unAutomovil.acelerar()
4 else:
5     unAutomovil.frenar()
```

Yendo más allá, los objetos también son valores, al mismo nivel que los enteros, strings, estructuras, etc. Por lo mismo, podemos incluso pasar un objeto *como parámetro* y *retornar* objetos. Por ejemplo, nuestros métodos `acelerar` y `frenar` pueden retornar como resultado el *mismo* objeto, lo que resulta en un patrón particular como el que sigue a continuación:

```
1 >>> unAutomovil.acelerar().acelerar().frenar().acelerar().frenar()
```

Esto es posible dado que al llamar a `acelerar` o `frenar`, el resultado de la llamada retorna la *misma* instancia de `unAutomovil`, a la cual se puede volver a llamar a los mismos métodos.

14.3 Múltiples instancias de una clase y estado de los objetos

Podemos crear distintas instancias de una clase como objetos. En nuestro ejemplo, podemos crear distintos automóviles:

```
1 >>> unAutomovil = Automovil()
2 >>> otroAutomovil = Automovil()
3 >>> # a hacerlos competir!
```

¿En qué se diferencian dos objetos instanciados de la misma clase? A simple vista no se ve mucha diferencia (ambos objetos pueden acelerar, frenar, encenderse, apagarse, etc.), pero lo que los caracteriza es lo que los diferencia. En lo que hemos visto hasta ahora, si el automóvil acelera

durante 30 segundos, podemos esperar que su nivel de gasolina disminuya. Como dijimos al comienzo, si queremos representar un automóvil por su color, su velocidad, etc. estamos diferenciándolos. Pero también podemos cambiar estos valores (como la gasolina). Estos valores constituyen el *estado* del objeto. Tal como en las estructuras mutables, el estado de un objeto también puede cambiar. Veamos un par de ejemplos.

Si queremos crear un automóvil de cierto color y cierta velocidad máxima, podríamos hacerlo al momento de instanciar el objeto:

```
1 >>> miAutomovil = Automovil(color="azul", velocidadMaxima=220)
```

Si queremos luego modificar estos valores (supongamos que *enchulamos* nuestro automóvil) también podemos hacerlo:

```
1 >>> miAutomovil.setColor("rojo")
2 >>> miAutomovil.setVelocidadMaxima(250)
```

14.4 Ejemplo: libreta de direcciones

Volvamos al ejemplo de la libreta de direcciones del capítulo de Mutación. Supongamos que tenemos dos clases: `Registro` y `Libreta`, con los cuales podemos crear objetos que representen registros en nuestra libreta, con nombre, teléfono y dirección; y una forma de crear distintas libretas con nombre:

```
1 >>> libretaPersonal = Libreta("personal")
2 >>> libretaTrabajo = Libreta("trabajo")
3 >>> registro1 = Registro(nombre="Juan Gonzalez", \
4     telefono="777-7777", direccion="Calle ABC 123")
5 >>> libretaTrabajo.agregarRegistro(registro1)
```

Hasta ahora hemos creado distintas libretas y un registro, y hemos dado un estado específico a cada libreta y registro. Así como antes teníamos una función para buscar un registro dado un nombre, también podemos hacer lo mismo usando nuestra representación en objetos, suponiendo que tenemos los métodos adecuados:

```
1 >>> john = libretaTrabajo.buscar("John")
2 >>> john.setTelefono("133")
3 >>> john.getTelefono()
4 "133"
```

Capítulo 15

Definición de Clases

En este capítulo estudiaremos cómo definir una clase en Python, cómo definir los campos de la clase, cómo definir la construcción de un objeto, y cómo definir métodos en la clase. Como ejemplo, implementaremos una clase que nos permita crear objetos para manejar fracciones (como en el capítulo de datos compuestos).

15.1 Clase

Para definir una clase en Python se utiliza la instrucción `class`. En esta instrucción se debe señalar el nombre que tendrá la clase. Por convención, los nombres de las clases comienzan con mayúscula. En este punto, y como parte de la receta de diseño, señalaremos los campos (también llamados *variables de instancia*) que tendrá la clase y sus tipos. Veremos una primera versión de nuestra clase para manejar fracciones, que denominaremos `FraccionV1`:

```
1 # Campos :
2 # numerador : int
3 # denominador : int
4 class FraccionV1:
```

15.1.1 Campos

Los campos de una clase son variables de estado que nos permiten almacenar información sobre los objetos de dicha clase. Para la clase `FraccionV1` necesitamos al menos dos campos: uno para almacenar el numerador de la fracción, y otro para almacenar el denominador de la fracción. Al ser variables de estado, su valor se puede modificar haciendo la asignación correspondiente al valor nuevo.

15.2 Constructor

El constructor es el primer método que uno debe definir en una clase. Este método se ejecuta cada vez que se crea un nuevo objeto de la clase. Usualmente, en este método es en donde se definen los campos de la clase y sus valores iniciales. Para la clase `FraccionV1`, el constructor es el siguiente:

```
1 # Constructor
2 def __init__(self, numerador = 0, denominador = 1):
3     # Inicializacion de campos
4     self.numerador = numerador
5     self.denominador = denominador
```

En Python, el método constructor siempre tiene el nombre `__init__`, y sólo se puede definir un constructor por clase. Todo método de una clase en Python (incluyendo al constructor) tiene como primer parámetro la palabra clave `self`, que es una referencia al objeto que se está creando, aunque cuando uno crea un objeto *no coloca nada para dicho parámetro*. Note que el constructor para la clase `FraccionV1` recibe además dos parámetros, el numerador y el denominador. Si el usuario no los especifica al crear el objeto, se especifica que esas variables tendrán los valores 0 y 1 por default. Por ejemplo:

```
1 >>> f = FraccionV1(1, 2) # crea la fraccion 1/2
2 >>> f = FraccionV1() # crea la fraccion 0/1
```

Dentro del constructor se definen e inicializan las dos variables de instancias: `self.numerador` y `self.denominador`. Note que es necesario anteponer `self`. cada vez que se desee accesar o modificar dichos campos, sino Python interpreta que el programador se está refiriendo a variables locales del método. Es usual definir e inicializar todos los campos dentro del constructor, aunque es posible agregar campos a la clase posteriormente, definiendo nuevas variables de estado en otros métodos de la clase.

15.3 Métodos

Los métodos de una clase se definen igual que las funciones en Python, con la diferencia que se definen dentro del contexto de una clase y deben tener como primer parámetro la referencia `self`. Note que este primer parámetro `self` no es parte del contrato del método. Por ejemplo, definamos un método para la clase `FraccionV1` que nos permita sumar dos fracciones:

```
1 # suma: Fraccion -> Fraccion
2 # devuelve la suma de la fraccion con otra fraccion
3 def suma(self, fraccion):
4     num = self.numerador * fraccion.denominador + \
5         fraccion.numerador * self.denominador
6     den = self.denominador * fraccion.denominador
7     return FraccionV1(num, den)
```

El método es muy similar a la función `suma` que implementamos en el capítulo de datos compuestos. Note `self` corresponde al objeto que invoca al método `suma` y `fraccion` corresponde al objeto que se pasó por parámetro al método. Por ejemplo, en el siguiente código:

```
1 f1 = FraccionV1(1, 2)
2 f2 = FraccionV1(5, 6)
3 f3 = f1.suma(f2)
```

el objeto `f1` corresponde a `self` en el método `suma`, y el objeto `f2` corresponde al parámetro `fraccion` en dicho método.

15.3.1 Métodos accesorios y mutadores

Los métodos de una clase se pueden dividir en dos categorías: accesorios (*accessors*) y mutadores (*mutators*). Los métodos accesorios son aquellos que sólo acceden al contenido de los campos del objeto, pero no los modifican. Por ejemplo, el método `suma` es un ejemplo de un método accesor. Por otra parte, los métodos mutadores son aquellos que modifican (o pueden modificar) los valores de los campos de la clase. Al igual que en las funciones con memoria que modifican el estado de una variable, debemos indicar el efecto que puede tener un método mutador sobre los campos de la clase.

Por ejemplo, veamos un método mutador que permite simplificar una fracción. Para esto, se debe modificar tanto el numerador como el denominador de la fracción, dividiendo ambos valores por el

APUNTE DE USO INTERNO PROHIBIDA SU DISTRIBUCIÓN

máximo común divisor. Suponiendo que disponemos de la función `mcd(x,y)` que calcula el máximo común divisor entre dos números enteros `x` e `y`, podemos implementar el método `simplificar` de la siguiente forma:

```

1      # simplificar: None -> Fraccion
2      # efecto: simplifica la fraccion, puede modificar los
3      # valores de los campos numerador y denominador
4      def simplificar(self):
5          valor = mcd(self.numerador, self.denominador)
6          if valor > 1:
7              self.numerador = self.numerador / valor
8              self.denominador = self.denominador / valor

```

Es muy típico definir en una clase métodos accesorios para obtener el valor de los distintos campos, y métodos mutadores para asignarles un nuevo valor. Por convención, los nombres de los métodos accesorios comienzan con *get*, y los mutadores comienzan con *set*. Para nuestra clase `FraccionV1`, los métodos correspondientes son los siguientes:

```

1      # getNumerador: None -> int
2      # devuelve el valor del campo numerador
3      def getNumerador(self):
4          return self.numerador
5
6      # getDenominador: None -> int
7      # devuelve el valor del campo denominador
8      def getDenominador(self):
9          return self.denominador
10
11     # setNumerador: int -> None
12     # efecto: modifica el valor del campo numerador
13     def setNumerador(self, numerador):
14         self.numerador = numerador
15
16     # setDenominador: int -> None
17     # efecto: modifica el valor del campo denominador
18     def setDenominador(self, denominador):
19         self.denominador = denominador

```

15.4 Receta de diseño de clases

La receta de diseño para clases consiste en los siguientes pasos:

- Antes de definir la clase, se identifican los campos que tendrá y sus tipos correspondientes.
- La definición de los métodos sigue las reglas habituales de la receta de diseño para funciones, pero los cuerpos de los métodos y los tests correspondientes quedan pendientes.
- Una vez terminada la definición de métodos, fuera de la clase se implementan los tests para todos los métodos. Esto es así porque es necesario crear objetos de la clase con los cuales invocar los objetos, y dependiendo de los valores de los campos de cada objeto se puede determinar la respuesta esperada a cada método.
- Finalmente, se implementan los cuerpos de los métodos, y luego se ejecutan los tests. Se corrigen los errores detectados en los tests, y se itera nuevamente hasta que todos los tests sean exitosos.

A continuación se presenta la implementación completa de la clase `FraccionV1`. Note que la función `mcd` no es un método de la clase (no tiene como primer parámetro `self`), sino una función auxiliar que debe ser declarada como `global` para poder ser utilizada por los métodos.

Contenido del archivo `FraccionV1.py`

```

1  # Campos :
2  # numerador : int
3  # denominador : int
4  class FraccionV1:
5
6      # Constructor
7      def __init__(self, numerador = 0, denominador = 1):
8          # Inicialización de campos
9          self.numerador = numerador
10         self.denominador = denominador
11
12         # getNumerador: None -> int
13         # devuelve el valor del campo numerador
14         def getNumerador(self):
15             return self.numerador
16
17         # getDenominador: None -> int
18         # devuelve el valor del campo denominador
19         def getDenominador(self):
20             return self.denominador
21
22         # setNumerador: int -> None
23         # efecto: modifica el valor del campo numerador
24         def setNumerador(self, numerador):
25             self.numerador = numerador
26
27         # setDenominador: int -> None
28         # efecto: modifica el valor del campo denominador
29         def setDenominador(self, denominador):
30             self.denominador = denominador
31
32         # toString: None -> str
33         # devuelve un string con la fraccion
34         def toString(self):
35             return str(self.numerador) + "/" + str(self.denominador)
36
37         # suma: Fraccion -> Fraccion
38         # devuelve la suma de la fraccion con otra fraccion
39         def suma(self, fraccion):
40             num = self.numerador * fraccion.denominador + \
41                 fraccion.numerador * self.denominador
42             den = self.denominador * fraccion.denominador
43             return FraccionV1(num, den)
44
45         # mcd: int int -> int

```

Contenido del archivo FraccionV1.py (cont)

```

46     # devuelve el maximo comun divisor entre dos numeros x e y
47     # ejemplo: mcd(12, 8) devuelve 4
48     global mcd
49     def mcd(x, y):
50         if x == y:
51             return x
52         elif x > y:
53             return mcd(x-y, y)
54         else:
55             return mcd(x, y-x)
56
57     # Test
58     assert mcd(12, 8) == 4
59
60     # simplificar: None -> Fraccion
61     # efecto: simplifica la fraccion, puede modificar los
62     # valores de los campos numerador y denominador
63     def simplificar(self):
64         valor = mcd(self.numerador, self.denominador)
65         if valor > 1:
66             self.numerador = self.numerador / valor
67             self.denominador = self.denominador / valor
68
69     # Tests
70     f1 = FraccionV1(1, 2)
71     f2 = FraccionV1(5, 6)
72     # Test de accesors
73     assert f1.getNumerador() == 1
74     assert f2.getDenominador() == 6
75     # Test de mutators
76     f2.setNumerador(3)
77     f2.setDenominador(4)
78     assert f2.getNumerador() == 3 and f2.getDenominador() == 4
79     # Test de metodo suma
80     # El siguiente test es incorrecto
81     # assert f1.suma(f2) == FraccionV1(10, 8)
82     # El siguiente test es correcto
83     f3 = f1.suma(f2)
84     assert f3.getNumerador() == 10 and f3.getDenominador() == 8
85     # Test de metodo toString
86     assert f3.toString() == "10/8"
87     # Test de metodo simplificar
88     f3.simplificar()
89     assert f3.getNumerador() == 5 and f3.getDenominador() == 4

```

La clase `FraccionV1` contiene métodos accesorios y mutadores. Si se desea implementar la clase sólo usando métodos accesorios (es decir, se opera con los objetos de forma puramente funcional), se puede hacer eliminando todo método mutador de la clase o modificándolo a una versión accesorio,

en donde el resultado se almacena en un nuevo objeto de la misma clase. Como detalle adicional, para evitar que un usuario fuera de la clase pueda modificar los campos de un objeto, en Python se pueden definir con nombres que comiencen con los caracteres `--` (dos caracteres de guión bajo), y esto los hace inaccesibles fuera de la clase (si uno intenta modificarlos, Python arroja el error `AttributeError`). La implementación de la clase `FraccionV2`, que sólo utiliza métodos accesorios es la siguiente:

Contenido del archivo `FraccionV2.py`

```

1  # Campos :
2  # numerador : int
3  # denominador : int
4  class FraccionV2:
5
6      # Constructor
7      def __init__(self, numerador = 0, denominador = 1):
8          # Inicializacion de campos
9          # campos invisibles al usuario
10         self.__numerador = numerador
11         self.__denominador = denominador
12
13         # getNumerador: None -> int
14         # devuelve el valor del campo numerador
15         def getNumerador(self):
16             return self.__numerador
17
18         # getDenominador: None -> int
19         # devuelve el valor del campo denominador
20         def getDenominador(self):
21             return self.__denominador
22
23         # toString: None -> str
24         # devuelve un string con la fraccion
25         def toString(self):
26             return str(self.__numerador) + "/" + str(self.__denominador)
27
28         # suma: Fraccion -> Fraccion
29         # devuelve la suma de la fraccion con otra fraccion
30         def suma(self, fraccion):
31             num = self.__numerador * fraccion.__denominador + \
32                 fraccion.__numerador * self.__denominador
33             den = self.__denominador * fraccion.__denominador
34             return FraccionV2(num, den)
35
36         # mcd: int int -> int
37         # devuelve el maximo comun divisor entre dos numeros x e y
38         # ejemplo: mcd(12, 8) devuelve 4
39         global mcd
40         def mcd(x, y):
41             if x == y:
42                 return x
43             elif x > y:

```

Contenido del archivo FraccionV2.py (cont)

```
44         return mcd(x-y, y)
45     else:
46         return mcd(x, y-x)
47
48     # Test
49     assert mcd(12, 8) == 4
50
51     # simplificar: None -> Fraccion
52     # devuelve la fraccion simplificada
53     def simplificar(self):
54         valor = mcd(self.__numerador, self.__denominador)
55         num = self.__numerador / valor
56         den = self.__denominador / valor
57         return FraccionV2(num, den)
58
59     # Tests
60     f1 = FraccionV2(1, 2)
61     f2 = FraccionV2(3, 4)
62     # Test de accesors
63     assert f1.getNumerador() == 1
64     assert f2.getDenominador() == 4
65     # Test de metodo suma
66     f3 = f1.suma(f2)
67     assert f3.getNumerador() == 10 and f3.getDenominador() == 8
68     # Test de metodo toString
69     assert f3.toString() == "10/8"
70     # Test de metodo simplificar
71     f4 = f3.simplificar()
72     assert f4.getNumerador() == 5 and f4.getDenominador() == 4
```

Capítulo 16

Interacciones entre Objetos¹

En capítulos anteriores hemos visto qué son los objetos y las clases, y cómo se implementan en Python. En particular, discutimos las nociones de campo, constructor y métodos cuando hablamos de definición de clases.

Ahora iremos un paso más adelante. Para construir aplicaciones interesantes, no es suficiente el construir objetos independientes. En efecto, nos interesaría que los objetos puedan combinarse entre sí, de tal manera que juntos puedan realizar una tarea común. En este capítulo desarrollaremos esta idea a través de una pequeña aplicación de ejemplo que involucra tres objetos y un conjunto de métodos que permitan cumplir con su tarea.

Consideremos un reloj digital. Este tipo de relojes tiene una pantalla en la cual se muestran las horas y los minutos, separados por el símbolo dos puntos (:). Así, estos relojes son capaces de mostrar la hora desde las 00:00 (medianoche) hasta las 23:59 (un minuto antes de medianoche).

16.1 Abstracción y modularidad con objetos

Una primera idea para implementar el reloj puede ser desarrollar una única clase. Después de todo, es precisamente este el enfoque que hemos seguido hasta ahora: cómo construir clases para desarrollar un trabajo. Sin embargo, inspirándonos en lo que ya sabemos sobre *abstracción y modularidad*, notamos que esta no es la mejor manera de proceder. La idea principal es identificar subcomponentes en el problema que se puedan descomponer en clases separadas. La razón de esto es poder manejar apropiadamente la complejidad. En efecto, mientras más grande es un problema, se vuelve cada vez más difícil llevar un registro de todos los detalles que hay que manejar al mismo tiempo.

La solución que usaremos para manejar la complejidad cuando desarrollamos programas usando objetos es la *abstracción*. Dividiremos el problema en subproblemas, y luego cada subproblema en sub-subproblemas hasta que los problemas individuales sean lo suficientemente pequeños y manejables como para poder desarrollarlos con una clase sencilla (a esto se le denomina *modularidad*). Una vez que hayamos resuelto uno de estos subproblemas, no nos preocuparemos más de los detalles de éste, sino que consideraremos esta solución como un elemento que podemos reutilizar en el subproblema siguiente. Típicamente, a esta estrategia la llamamos *dividir-y-conquistar* o *dividir-para-reinar*.

Así, la modularidad y la abstracción se complementan entre sí. La modularidad es el proceso de dividir problemas grandes en partes más pequeñas, mientras que la abstracción es la habilidad de

¹Parte de este texto fue traducido al español y adaptado de: Chapter 3 “Object interaction”, in David J. Barnes and Michael Kölling: *Objects First with Java - A Practical Introduction using BlueJ*, Fifth Edition, Prentice Hall.

ignorar detalles para enfocarse en el plano general.

Los mismos principios de modularidad y abstracción que acabamos de revisar se aplican en el desarrollo de software orientado a objetos. Para ayudarnos a mantener una visión general en programas complejos, intentamos identificar subcomponentes que podamos programar como entidades independientes. Luego, tratamos de usar esas subcomponentes como si fueran unidades simples, sin tener que preocuparnos de su complejidad interna.

En la programación orientada a objetos, estas componentes y subcomponentes son objetos. Si intentáramos por ejemplo construir un auto como si fuera un software, usando un lenguaje orientado a objetos, lo que haríamos sería construir objetos separados para el motor, la caja de cambios, una rueda, un asiento, entre otros, en lugar de modelar el auto como un objeto simple y monolítico. El identificar qué tipo de objetos (y qué clases) se deben incluir en un sistema para un problema dado no siempre es una tarea sencilla.

Volvamos al ejemplo del reloj digital. Usando los conceptos de abstracción que hemos revisado, nos gustaría encontrar la mejor manera de escribir una o más clases para implementarlo. Una forma de ver el problema es considerar al reloj como una pantalla con cuatro dígitos (dos para las horas y dos para los minutos). Si ahora realizamos una abstracción a más alto nivel, podemos ver al reloj como dos entidades distintas de dos dígitos cada una (un par para representar las horas, y otro par para los minutos). Así, un par empieza en 0, aumenta en 1 cada hora, y vuelve a 0 cuando alcanza su límite 23. El otro par vuelve a 0 cuando su valor alcanza el límite 59. Lo similar en el comportamiento de estas dos entidades nos da para pensar que podemos abstraer aún más el problema, y por ende, dejar de ver al reloj como una combinación de horas y minutos.

En efecto, podemos pensar que el reloj está formado por dos objetos que pueden mostrar valores enteros que comienzan en 0 hasta cierto límite. Este valor puede aumentar, pero, si alcanza el límite, se reinicia a 0. Ahora sí tenemos un nivel de abstracción apropiado que podemos representar como una clase: un par de números programables. Así, para programar la pantalla del reloj, primero debemos implementar una clase para manejar un par de números, luego darle un método para obtener su valor, y dos métodos para asignar un valor y aumentarlo. Una vez que hayamos definido esta clase, simplemente bastará con crear dos objetos de esta clase (cada cual con diferentes límites) para así construir el reloj completo.

16.2 Diagramas de clases y objetos

Tal como lo discutimos anteriormente, para programar el reloj, necesitamos primero construir una representación para un par de números. Este par tiene que almacenar naturalmente dos valores: uno es el límite hasta el cual se puede contar sin tener que reiniciar a cero; el otro es el valor actual. Representaremos estos dos valores como campos enteros en la clase `ParDeNumeros`:

```
1 # Campos:
2 # limite: int
3 # valor: int
4 class ParDeNumeros:
5     ...
```

Veremos más adelante los detalles de implementación de esta clase. Primero, asumamos que podemos construir esta clase, y pensemos un poco más en cómo podemos organizar el reloj completo.

Nos gustaría poder construir el reloj, a partir de un objeto que tenga internamente dos pares de números (uno para las horas y otro para los minutos). Así, cada uno de estos pares de números sería

un campo para un objeto de la clase `Reloj`:

```
1 # Campos:  
2 # horas: ParDeNumeros  
3 # minutos: ParDeNumeros  
4 class Reloj:  
5     ...
```

La estructura de objetos descrita puede visualizarse usando el siguiente *diagrama de objetos*. En este diagrama apreciamos que un objeto de la clase `Reloj` se instancia utilizando dos objetos de la clase `ParDeNumeros`.

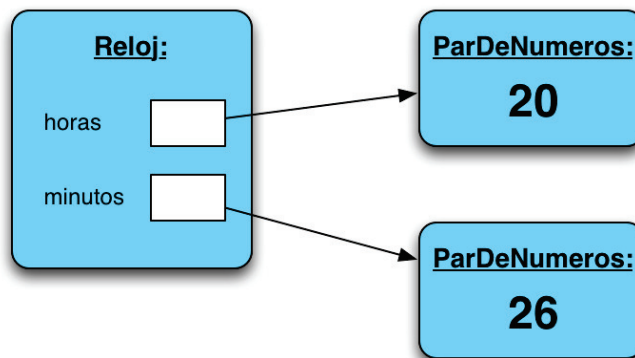


Figura 16.1: Diagrama de objetos

Asimismo, la siguiente figura muestra el *diagrama de clases* que modela el problema del reloj digital.

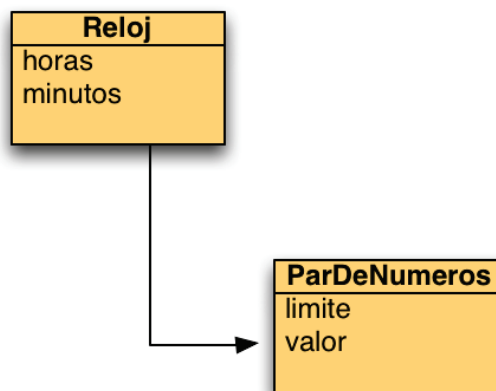


Figura 16.2: Diagrama de clases

Notemos que el diagrama de clases muestra únicamente dos clases, mientras que el diagrama de objetos muestra tres objetos. Esto se debe al hecho que podemos crear más de un objeto desde una misma clase. En este caso, creamos dos objetos `ParDeNumeros` desde la clase `ParDeNumeros`.

Estos dos diagramas ofrecen distintas vistas de la misma aplicación. El diagrama de clases muestra la *vista estática*. Muestra qué es lo que tenemos al momento de escribir el programa. Así, tenemos dos clases, y la flecha entre ellas indica que la clase `Reloj` hace uso de la clase `ParDeNumeros`. En otras palabras, en el código fuente de la clase `Reloj` aparecerá una o más referencias a la clase `ParDeNumeros`.

Para comenzar el programa, crearemos un objeto de la clase `Reloj`. Programaremos así la pantalla del reloj digital de tal manera que cree automáticamente dos objetos `ParDeNumeros`. En efecto, el diagrama de objetos muestra esta situación en tiempo de ejecución, es decir, cuando la aplicación está corriendo. Esto también recibe el nombre de *vista dinámica*.

El diagrama de objetos también muestra otro detalle importante: cuando una variable almacena un objeto, el objeto no es almacenado directamente en la variable, sino que una *referencia* al objeto es almacenado en la variable. En el diagrama, la variable se representa como una caja blanca, y el objeto es mostrado como una flecha. El objeto referenciado es almacenado fuera del objeto que referencia, y la referencia de objetos enlaza a ambos.

Ahora analizaremos la implementación del reloj digital. Primero, debemos programar la clase `ParDeNumeros`. En esta clase, notamos los dos campos que discutimos más arriba, un constructor, y cuatro métodos. El constructor recibe el valor del límite como parámetro. Así, por ejemplo, si recibe 24 como parámetro, el valor se reiniciará a 0 cuando se llegue a ese valor. De esta forma, el rango para el valor que se puede almacenar en este caso va de 0 a 23. Con esto, recordemos que podemos definir correctamente las horas y los minutos a manejar en el reloj: para las horas usamos un límite de 24, y para los minutos, un límite de 60.

```
1 # Campos:
2 # limite: int
3 # valor: int
4 class ParDeNumeros:
5
6     # Constructor: crea un objeto que almacena dos numeros y se reinicia
7     # a cero cuando se sobrepasa el limite
8     def init (self, limite):
9         self.limite = limite
10        self.valor = 0
11
12    # getValor: None -> int
13    # Retorna el valor actual
14    def getValor(self):
15        return self.valor
16
17    # setValor: int -> None
18    # efecto: Reemplaza el valor del par al nuevo valor indicado.
19    # Si el nuevo valor es menor que cero, o sobre el limite,
20    # no hacer nada.
21    def setValor(self, nuevoValor):
22        if (nuevoValor >= 0) and (nuevoValor < self.limite):
23            self.valor = nuevoValor
24
25    # toString: -> str
26    # Retorna el valor almacenado en el par, esto es, un string que
27    # contiene los numeros del par; si el valor es menor que diez, se le
```

```

28  # debe anteponer un cero
29  def toString(self):
30      if self.valor < 10:
31          return "0" + str(self.valor)
32      else:
33          return str(self.valor)
34
35  # aumentar: None -> None
36  # efecto: Aumenta en una unidad el valor almacenado en el par,
37  # reiniciando a cero si se sobrepasa el limite
38  def aumentar(self):
39      self.valor = (self.valor + 1) % self.limite

```

Ahora que ya vimos cómo construir una clase que define un número de dos dígitos, miraremos más en detalle a la clase `Reloj`.

```

1  # Campos:
2  # horas: ParDeNumeros
3  # minutos: ParDeNumeros
4  # pantalla: str
5  class Reloj:
6
7      # Constructor: crea un objeto reloj. Si no recibe parametros,
8      # inicia el reloj a las 00:00; si no, a la hora indicada
9      def init (self, horas=0, minutos=0):
10         self.horas = ParDeNumeros(24)
11         self.minutos = ParDeNumeros(60)
12         self.setReloj(horas, minutos)
13
14      # tic: None -> None
15      # Se debe llamar cada minuto y hace que el reloj avance un minuto
16      def tic(self):
17         self.minutos.aumentar()
18         if self.minutos.getValor() == 0:
19             self.horas.aumentar()
20             self.actualizarPantalla()
21
22      # setReloj: int int -> None
23      # efecto: Fija la hora del reloj a la hora y minuto especificados
24      def setReloj(self, hora, minuto):
25         self.horas.setValor(hora)
26         self.minutos.setValor(minuto)
27         self.actualizarPantalla()
28
29      # getHora: None -> str
30      # Devuelve la hora actual del reloj en el formato HH:MM
31      def getHora(self):
32         return self.pantalla
33
34      # actualizarPantalla: None -> None
35      # efecto: Actualiza el string interno que lleva cuenta de la hora
36      # actual

```

```

37 def actualizarPantalla(self):
38     self.pantalla = self.horas.toString() + ":" + self.minutos.toString()

```

16.3 Objetos que crean objetos

Una pregunta que podemos hacernos es: ¿de dónde salen estos tres objetos? Cuando queramos usar un reloj digital, nosotros crearemos un objeto `Reloj`. Suponemos que nuestro reloj digital tiene horas y minutos. Luego, por el sólo hecho de crear un reloj digital, nosotros esperamos que se hayan creado implícitamente dos objetos de la clase `ParDeNumeros`, uno para las horas y otro para los minutos.

Como los implementadores de la clase `Reloj`, debemos asegurarnos que esto realmente pase. Para esto, simplemente escribimos código en el constructor de `Reloj` que crea y guarda dos objetos de `ParDeNumeros`. Ya que el constructor es automáticamente llamado cuando un objeto `Reloj` es creado, los objetos `ParDeNumeros` serán creados de manera automática también. Este es el código del constructor de `Reloj` que hace este trabajo:

```

1 # Campos:
2 # horas: ParDeNumeros
3 # minutos: ParDeNumeros

```

Otros campos omitidos

```

1 class Reloj:
2
3     # Constructor: crea un objeto reloj. Si no recibe parametros,
4     # inicia el reloj a las 00:00; si no, a la hora indicada
5     def init (self, horas=0, minutos=0):
6         self.horas = ParDeNumeros(24)
7         self.minutos = ParDeNumeros(60)
8         self.setReloj(horas, minutos)

```

Métodos omitidos

Cada una de estas dos líneas en el constructor crea un nuevo objeto `ParDeNumeros` y los asignan a una variable. Como ya hemos visto, la sintaxis para crear un nuevo objeto es:

NombreDeClase(lista-de-parámetros)

La construcción de un objeto hace dos cosas:

1. Crea un nuevo objeto de la clase llamada (en este caso `Reloj`)
2. Ejecuta el constructor de la clase

Si el constructor de la clase está definido para tener parámetros, entonces deben ser suministrados al crear el objeto. Por ejemplo, el constructor de la clase `ParDeNumeros` fue definido para esperar un parámetro del tipo entero:

```

1 class ParDeNumeros:
2     def init (self, limite):

```

Es importante notar que en Python la definición del constructor de una clase siempre contiene como primer parámetro `self` que no se considera como un argumento al momento de crear un objeto. Así, para crear un objeto de la clase `ParDeNumeros` debemos proveer de un parámetro de tipo entero:

```
1 ParDeNumeros(24)
```

Luego, con el constructor de la clase `Reloj` hemos conseguido lo que queríamos: cuando creamos un objeto de esta clase, su constructor será ejecutado automáticamente y creará dos objetos de la clase `ParDeNumeros`. Es decir, este objeto crea a su vez otros dos objetos cuando es creado y nuestra clase `Reloj` está lista para ser usada.

16.4 Llamadas a métodos

Un método puede llamar a otros métodos dentro de una misma clase como parte de su implementación. Esto se denomina *llamada de método interna*. Por otra parte, un método puede llamar a métodos de otros objetos usando un punto como notación. Esto se denomina *llamada de método externa*. En esta sección revisaremos ambos tipos de llamadas.

Llamadas de métodos internas

La última línea del método `tic` de la clase `Reloj` contiene la siguiente declaración:

```
1 self.actualizarPantalla()
```

Esta declaración es una *llamada a un método*. Como hemos visto hasta el momento, la clase `Reloj` tiene un método con la siguiente firma:

```
1 def actualizarPantalla(self):
```

La llamada al método de arriba invoca a este método. Ya que este método está en la misma clase que la llamada al método (en `tic`), también la denominamos *llamada de método interna*. Este tipo de llamadas tienen la siguiente sintaxis:

```
1 self.nombreDelMetodo( lista-de-parametros )
```

En nuestro ejemplo, el método no tienen ningún parámetro, así que la lista de parámetros está vacía. Esto está indicado por el par de paréntesis sin nada dentro de ellos.

Cuando en el código se encuentra una llamada a un método, el método correspondiente es ejecutado, y la ejecución retorna al método en donde fue ejecutada la llamada y continúa con la siguiente instrucción. Para que la firma (el contrato) de un método corresponda a la llamada de un método, tanto el nombre del método como la lista de parámetros deben corresponder. Aquí, ambas listas de parámetros están vacías (puesto que el parámetro `self` no se considera), por lo que corresponden. Esta necesidad de corresponder tanto con el nombre como con la lista de parámetros es importante puesto que la llamada de un método con un número de parámetros equivocados falla, como en el caso de funciones.

En nuestro ejemplo, el propósito de esta llamada al método es actualizar el string a desplegar en la pantalla. Luego que ambos objetos de la clase `ParDeNumeros` han sido creados, el string a desplegar es asignado para mostrar el tiempo indicado por el número de los objetos `ParDeNumeros`. La implementación del método `actualizarPantalla` será explicado a continuación.

Llamadas de métodos externas

Examinemos el siguiente método: `tic`. Su definición está dada por:

```

1 # tic: None -> None
2 # Se debe llamar cada minuto y hace que el reloj avance un minuto
3 def tic(self):
4     self.minutos.aumentar()
5     if self.minutos.getValor() == 0:
6         self.horas.aumentar()
7     self.actualizarPantalla()

```

Cuando la pantalla está conectada a un reloj de verdad, este método debería ser llamado cada 60 segundos por el timer electrónico del reloj. Por ahora, nosotros mismos haremos esta llamada en forma manual para probar nuestra clase `Reloj`. Cuando el método `tic` es llamado, la primera sentencia que es ejecutada es:

```

1 self.minutos.aumentar()

```

Esta declaración llama al método `aumentar` del objeto `minutos`. Luego, cuando uno de los métodos del objeto `Reloj` es llamado, a su vez él llama a un método de otro objeto para hacer parte del trabajo. Una llamada a un método de otro objeto se denomina *llamada de método externa*. La sintaxis de una llamada de este tipo esta dada por:

```

1 objeto.nombreDeMetodo( lista-de-parametros )

```

Esta notación es conocida como *notación de punto*. Consiste en el nombre de un objeto, un punto, el nombre del método a llamar, y los parámetros de la llamada. Es particularmente importante apreciar que aquí hablamos del nombre de un *objeto* y no del nombre de la clase: Usamos el nombre `minutos` en vez de `ParDeNumeros`.

El método `tic` tiene una sentencia `if` para comprobar que las horas también deben aumentar cuando pasan los 60 minutos. Como parte de esta condición se llama a otro método del objeto `minutos`: `getValor`. Este método retorna el valor actual de los minutos. Si el valor es cero, entonces sabemos que ya han pasado 60 minutos por lo que debemos incrementar las horas. Por otra parte, si el valor no es cero, entonces hemos terminado, puesto que no debemos aumentar las horas. Luego, la declaración `if` no necesita la parte `else`.

Ahora debemos poder entender los tres métodos que nos restan de la clase `Reloj`. El método `setReloj` toma dos parámetros –la hora y el minuto– y asigna el reloj con el tiempo especificado. Mirando al cuerpo del método, podemos ver que esto lo hace llamando a los métodos `setValor` de ambos objetos `ParDeNumeros` (uno para las horas y uno para los minutos). Luego, éste llama a `actualizarPantalla` para actualizar el string de la pantalla.

El método `getHora` es trivial, dado que sólo retorna el string actual de la pantalla. Ya que siempre mantenemos el string de la pantalla actualizado, es todo lo que se debe hacer ahí.

Finalmente, el método `actualizarPantalla` es el responsable de actualizar el string de la pantalla para que éste refleje el tiempo representado por los dos objetos `ParDeNumeros`. Este método es llamado cada vez que el tiempo del reloj cambia. Trabaja llamando a los métodos `getValor` de cada uno de los objetos `ParDeNumeros`. Estos métodos retornan el valor de cada par de número por separado, y luego los usa para crear el string de la pantalla con estos dos valores con una coma de separación.

16.5 Testing de clases

La tarea de prueba de programas orientados al objeto es más compleja que probar programas funcionales, por el hecho que los objetos contienen variables de estado mutables, y que los métodos de un objetos se usan en combinación. Por estas dos razones, un test usualmente necesita definir un escenario de prueba más largo que una simple llamada a una función.

Ahora presentamos una primera manera de probar clases sencillas; volveremos al tema de las pruebas más adelante. La idea es que cada clase de un programa pueden tener una clase de prueba que se encarga de: (1) crear objetos de la clase a probar, y poner estos objetos en estados que queremos probar; (2) ejercitar la funcionalidad de dichos objetos con varias secuencias de métodos; y (3) verificar que el comportamiento es correcto.

Vamos a ver dos ejemplos. El primero es el test de la clase `ParDeNumeros`, donde tenemos que probar que los números aumentan hasta llegar al límite, y que la representación textual de dichos números siempre tiene dos caracteres. Esto se puede hacer de la siguiente manera:

```
1  # Para simplificar la implementacion de los tests,
2  # este codigo se incluye en el archivo donde se
3  # encuentra la definicion de la clase ParDeNumeros
4  class TestParDeNumeros:
5
6      def __init__(self):
7          # crear un objeto con estado interesante
8          self.par = ParDeNumeros(3)
9
10     def test(self):
11         # ejercitar funcionalidad,
12         # y verificar el comportamiento
13         assert self.par.getValor() == 0
14         self.par.aumentar()
15         assert self.par.getValor() == 1
16         self.par.aumentar()
17         assert self.par.getValor() == 2
18         self.par.aumentar()
19         assert self.par.getValor() == 0
20         self.par.aumentar()
21         assert self.par.toString() == "01"
22
23     # ejecucion del test
24     test = TestParDeNumeros()
25     test.test()
```

Observe que este escenario de test es más complejo que el test de una función única, sin efecto de borde. Seguimos con el ejemplo del test del Reloj, que aún más complejo, dado que tiene que probar que al avanzar los minutos se cambia de minuto y de hora, según el caso. Además, la pantalla tiene que tener el formato correcto.

```
1  class TestReloj:
2
3      def __init__(self):
4          # crear un objeto con estado interesante
5          self.reloj = Reloj(23,58)
6
```

```
7  def test(self):
8      # ejercitar funcionalidad,
9      # y verificar el comportamiento
10     assert self.reloj.getHora() == "23:58"
11     self.reloj.tic()
12     assert self.reloj.getHora() == "23:59"
13     self.reloj.tic()
14     assert self.reloj.getHora() == "00:00"
15     for i in range(60):
16         self.reloj.tic()
17         assert self.reloj.getHora() == "01:00"
18         self.reloj.tic()
19         assert self.reloj.getHora() == "01:01"
20
21 test = TestReloj()
22 test.test()
```

Capítulo 17

Diseño de Clases¹

En este capítulo veremos algunos de los factores que influyen en el diseño de una clase. ¿Qué hace que una clase esté bien o mal diseñada? Escribir una buena clase puede tomar más esfuerzo en el corto plazo que escribir una mala clase. Sin embargo, en el largo plazo ese esfuerzo extra es justificado con frecuencia. Para ayudarnos a escribir buenas clases existen algunos principios que podemos seguir. En particular, en este capítulo introduciremos la visión de que el diseño de una clase debe ser basado en responsabilidades, y que las clases deben encapsular sus datos.

17.1 Introducción

Es posible implementar una aplicación y conseguir que realice su tarea con clases mal diseñadas. El hecho de ejecutar una aplicación terminada usualmente no indica si está bien estructurada o no.

EL problema típicamente aparece cuando un programador de mantenimiento desea hacer algunos cambios a la aplicación existente. Si, por ejemplo, un programador intenta arreglar un *bug* o quiere agregar nuevas funcionalidades a un programa existente, una tarea que debería ser fácil y obvia con clases bien diseñadas podrían ser muy difíciles de realizar y pueden involucrar una gran cantidad de trabajo si las clases están mal diseñadas.

En aplicaciones grandes, este efecto ocurre en etapas tempranas, durante la implementación original. Si la implementación parte con una mala estructura, terminar la aplicación puede convertirse en una tarea en extremo compleja, o imposible de realizar, o que tenga *bugs*, o tome mucho más tiempo para construirse que lo necesario. En el mundo laboral, las compañías que desarrollan software usualmente mantienen, extienden y venden una aplicación a lo largo de muchos años. No es poco común que una implementación de software que se puede comprar en una tienda hoy haya comenzado a construirse hace más de diez años. En esta situación, una compañía de software no puede permitirse tener código mal estructurado.

Dado que muchos de los efectos de una clase mal diseñada se hacen más evidentes cuando tratamos de adaptarla o extender la aplicación a la que pertenece, en este capítulo haremos exactamente eso: utilizaremos como ejemplo el juego *mundo-de-zuul*. Para esto, ocuparemos una implementación mucho más simple y rudimentaria del juego de aventura basado en texto. En su estado original, el juego no es demasiado ambicioso, de hecho está incompleto. Para el final del capítulo, sin embargo, estarás en una posición de ejercitar tu imaginación, diseñar tu propio juego y hacerlo realmente divertido e

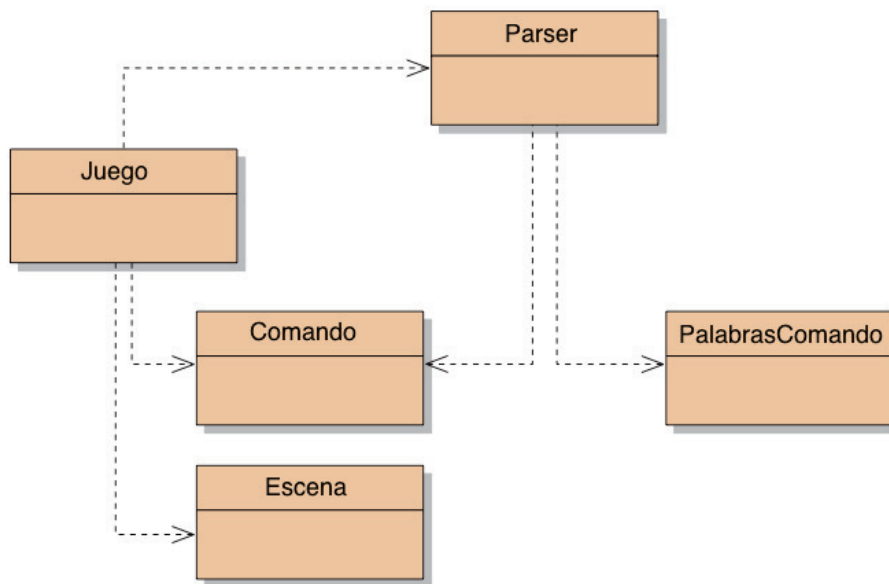
¹Parte de este texto fue traducido al español y adaptado de: Chapter 6 “Designing classes”, in David J. Barnes and Michael Kölling: *Objects First with Java - A Practical Introduction using BlueJ*, Fifth Edition, Prentice Hall.

interesante.

mundo-de-zuul Nuestro juego *mundo-de-zuul* está modelado a partir del juego *Aventura* original que fue desarrollado a principios de los 70' por Will Crowther y extendido por Don Woods. El juego original es también conocido como *Aventura Cueva Colosal*. Este fue un juego sofisticado y maravillosamente imaginativo para su tiempo, que incluía encontrar el camino en un complejo sistema de una cueva, encontrando tesoros escondidos, usando palabras secretas y otros misterios, para alcanzar el máximo de puntaje posible.

Mientras trabajamos en extender la aplicación original, aprovecharemos la oportunidad de discutir aspectos del diseño de clases existentes. Veremos que la implementación contiene ejemplos de mal diseño de clases, y podremos ver cómo esto impacta en nuestras tareas y cómo lo podemos arreglar.

Para partir, analizaremos la estructura de clases de la primera versión de nuestro juego que se observa a continuación:



Como podemos observar, el proyecto se compone de cinco clases. Estas son: **Parser**, **PalabrasComando**, **Comando**, **Escena** y **Juego**. El resumen de lo que cada una de estas clases hace se muestra a continuación:

- **PalabrasComando**: Define todos los comandos válidos en el juego. Esto lo hace almacenando un arreglo de strings que representan las palabras comando.
- **Parser**: El **Parser** lee las líneas desde la consola e intenta interpretarlas como comandos. Crea objetos de la clase **Comando** que representan el comando que fue ingresado.
- **Comando**: Un objeto **Comando** representa un comando que fue ingresado por el usuario. Tiene métodos que nos permite comprobar fácilmente si el comando ingresado es válido, y obtener la primera y segunda palabra que definen el comando como strings separados.

- *Escena*: Representa la ubicación del juego. Las escenas pueden tener salidas o llevar a otras escenas.
- *Juego*: La clase **Juego** es la clase principal del juego. Inicializa el juego y luego entra en un ciclo infinito que lee y ejecuta comandos. También contiene el código que implementa cada comando del usuario.

17.2 Introducción al acoplamiento y la cohesión

Si vamos a justificar nuestra afirmación de que algunos diseños son mejores que otros, debemos definir algunos términos que nos permitan discutir los factores que consideramos importantes en el diseño de una clase. Dos términos centrales cuando hablamos de la calidad del diseño de una clase son *acoplamiento* y *cohesión*.

El término *acoplamiento* se refiere a la interconexión de clases. Ya hemos discutido en capítulos anteriores acerca que nuestro objetivo es diseñar un conjunto de clases que cooperen entre sí y que se comuniquen a través de interfaces bien definidas. El grado de acoplamiento indica qué tan estrechamente estas clases están conectadas. Nos esforzamos para tener un bajo nivel de acoplamiento entre clases, o *acoplamiento holgado*.

El grado de acoplamiento determina qué tan difícil es hacer cambios a una aplicación. En un estructura con clases estrechamente acopladas, un cambio en una clase puede significar el cambio en muchas otras clases. Esto es lo que tratamos de evitar, puesto que el efecto de hacer un cambio pronto se expande a través de la aplicación completa. Además, encontrar todos los lugares donde es necesario hacer cambios y hacerlos puede ser difícil y consumir mucho tiempo.

Por otra parte, en un sistema con bajo nivel de acoplamiento frecuentemente podemos hacer cambios sin hacer ningún cambio a otras clases, y la aplicación aún funcionará. Discutiremos ejemplos de altos y bajos niveles de acoplamiento en este capítulo.

El término *cohesión* se relaciona con el número y diversidad de tareas que una sola unidad es responsable en la aplicación. La cohesión es relevante para unidades de una sola clase y métodos individuales.

Idealmente, una unidad debe ser responsable de una tarea cohesiva (esto es, una tarea que puede ser vista como una unidad lógica). Un método debe implementar una operación lógica, y una clase debe representar un tipo de entidad. La razón principal detrás de este principio es el reuso: si un método o una clase es responsable de solo una tarea bien definida, entonces es más probable de que sea usada de nuevo en un contexto diferente. Una ventaja complementaria a seguir este principio es que cuando un cambio es necesario para algún aspecto de la aplicación, es más probable que encontremos que la piezas relevantes para hacerlo estén ubicadas en un solo lugar.

17.3 Duplicación de código

La duplicación de código es un indicador de un mal diseño. La clase **Juego**, que se muestra a continuación, contiene un caso de duplicación. El problema con duplicación de código es que un cambio en una versión debe ser aplicado en todos los lugares que representen el mismo código para evitar inconsistencias. Esto aumenta la cantidad de trabajo que un programador de mantenimiento debe hacer, e introduce el peligro de insertar *bugs*. Es muy fácil asumir que el trabajo está hecho, pero no hay nada indicando que una segunda copia de este código exista, pudiendo mantenerse sin cambios

de manera incorrecta.

```

1 class Juego:
2
3     # ... Código omitido...
4     def crearEscenas(self):
5         afuera = Escena("Afuera de la entrada principal de la universidad")
6         auditorio = Escena("Sala de conferencias")
7         pub = Escena("En el pub del campus")
8         lab = Escena("En un laboratorio de computacion")
9         oficina = Escena("En la oficina principal de computacion")
10
11     # Inicializar las salidas de la escena
12     afuera.setSalidas(None, auditorio, lab, pub)
13     auditorio.setSalidas(None, None, None, afuera)
14     pub.setSalidas(afuera, oficina, None, None,)
15     oficina.setSalidas(None, None, None, lab)
16     self.escenaActual = afuera
17
18     # ... Código omitido...
19
20     # Imprime en pantalla el mensaje de bienvenida para el jugador
21     def imprimeBienvenida(self):
22         print ''
23         print 'Bienvenido al Mundo de Zuul'
24         print 'Zuul es un nuevo e increíblemente aburrido juego de aventura'
25         print ''
26         print 'Tu estas en ' + self.escenaActual.getDescripcion()
27         print 'Salidas: '
28         if self.escenaActual.salidaNorte != None
29             print 'norte',
30         if self.escenaActual.salidaEste != None
31             print 'este',
32         if self.escenaActual.salidaSur != None
33             print 'sur',
34         if self.escenaActual.salidaOeste != None
35             print 'oeste',
36         print ''
37
38     # ... Código omitido...
39
40     # Intentar ir en una direccion.
41     # Si hay una salida, entrar a la nueva escena; sino imprimir mensaje de error
42     def irAEscena(self, comando):
43         if not comando.tieneSegundoMundo():
44             # Si no hay segundo mundo
45             # no sabemos donde ir
46             print 'Ir a donde?'
47             return
48         direccion = comando.getSegundoMundo()
49         # Intentar salir de la escena

```

```

50     sgteEscena = None
51     if direccion.equals('norte'):
52         sgteEscena = self.escenaActual.salidaNorte
53     if direccion.equals('este'):
54         sgteEscena = self.escenaActual.salidaEste
55     if direccion.equals('sur'):
56         sgteEscena = self.escenaActual.salidaSur
57     if direccion.equals('oeste'):
58         sgteEscena = self.escenaActual.salidaOeste
59     if sgteEscena == None:
60         print 'No hay puerta!'
61     else:
62         self.escenaActual = sgteEscena
63         print 'Tu estas en ' + self.escenaActual.getDescripcion()
64         print 'Salidas: '
65         if self.escenaActual.salidaNorte != None:
66             print 'norte',
67         if self.escenaActual.salidaEste != None:
68             print 'este',
69         if self.escenaActual.salidaSur != None:
70             print 'sur',
71         if self.escenaActual.salidaOeste != None:
72             print 'oeste',
73         print ''
74
75     # ... Código omitido...

```

Ambos métodos `imprimeBienvenida` y `irAEscena` contienen las siguientes líneas de código:

```

1 print 'Tu estas en ' + self.escenaActual.getDescripcion()
2 print 'Salidas: '
3 if self.escenaActual.salidaNorte != None
4     print 'norte',
5 if self.escenaActual.salidaEste != None
6     print 'este',
7 if self.escenaActual.salidaSur != None
8     print 'sur',
9 if self.escenaActual.salidaOeste != None
10    print 'oeste',
11 print ''

```

La duplicación de código es usualmente un síntoma de cohesión. El problema aquí tiene sus raíces en el hecho que ambos métodos hacen dos cosas: `imprimeBienvenida` imprime el mensaje de bienvenida e información acerca de la ubicación actual, mientras que `irAEscena` cambia la ubicación actual y luego imprime la información de la (nueva) ubicación actual.

Ambos métodos imprimen información acerca de la ubicación actual, pero ninguno de ellos puede llamar al otro puesto que hacen otras cosas además. *Esto es mal diseño.*

Un mejor diseño sería usar un método separado y más cohesivo, cuya única tarea es imprimir la información de la ubicación actual. De esta forma, ambos métodos `imprimeBienvenida` y `irAEscena` pueden llamar a este método cuando necesitan imprimir la información de la escena actual en la que se encuentra el usuario. De esta manera, evitamos escribir el código dos veces, y cuando necesitemos

cambiarlo, sólo debemos hacerlo una vez.

```

1 def imprimeInfoUbicacion(self):
2     print 'Tu estas en ' + self.escenaActual.getDescripcion()
3     print 'Salidas: '
4     if self.escenaActual.salidaNorte != None
5         print 'norte',
6     if self.escenaActual.salidaEste != None
7         print 'este',
8     if self.escenaActual.salidaSur != None
9         print 'sur',
10    if self.escenaActual.salidaOeste != None
11        print 'oeste',
12    print ''

```

17.4 Acoplamiento

Consideremos ahora la posibilidad de agregar dos nuevas salidas: *arriba* y *abajo*. El hecho que en nuestro código existan tantos lugares en donde todas las salidas están enumeradas es un síntoma de un diseño de clase pobre. En los métodos `irAEscena` y `imprimeInfoUbicacion` hay expresiones `if` por cada salida de la variable del tipo `Escena` que estamos analizando. Esta decisión de diseño ahora nos crea trabajo: cuando agregamos nuevas salidas, debemos encontrar todos estos lugares y agregar nuevos casos y expresiones `if` asociadas. ¡Imagina el efecto de esto si decidimos usar direcciones como noroeste, sudeste, etc!

Veamos la implementación del método `setSalidas` de la clase `Escena`:

```

1 class Escena:
2
3     #... Codigo omitido...
4
5     def setSalidas(self, norte, este, sur, oeste):
6         if norte != None:
7             self.norte = norte
8         if este != None:
9             self.este = este
10        if sur != None:
11            self.sur = sur
12        if oeste != None:
13            self.oeste = oeste

```

Para mejorar esta situación, utilizaremos un diccionario que almacene todas las salidas en vez de guardarlas en variables separadas. Al hacer esto, deberíamos poder escribir código que pueda manejar cualquier número de salidas sin la necesidad de hacer demasiadas modificaciones en el código. El diccionario contendrá una asociación entre el nombre de la dirección (por ejemplo, `'norte'`) y la escena que está contigua en esa dirección (un objeto de tipo `Escena`). Entonces, cada entrada tiene un string como la llave y un objeto `Escena` como el valor.

Este es un cambio con respecto a cómo una escena almacena información internamente acerca de las escenas vecinas. Teóricamente, este cambio debería afectar solo la *implementación* de la clase `Escena`

(el *cómo* la información de las salidas es almacenada), no la *interfaz* (el *qué* almacena la escena).

Idealmente, cuando sólo la implementación de una clase cambia, otras clases no deberían verse afectadas. Esto significaría tener un bajo nivel de acoplamiento.

En nuestro ejemplo, esto no funciona. Si sacamos las variables que representan las salidas en la clase **Escena** y las reemplazamos con un diccionario, el juego no podrá ejecutarse de nuevo. Esto es puesto que existen demasiadas referencias a las variables de las salidas de una escena, y todas causan error cuando se introduce el nuevo cambio.

Vemos que acá tenemos un caso de un alto nivel de acoplamiento. Para limpiar esto, las desacoplaremos antes de introducir el diccionario.

17.4.1 Usar encapsulamiento para reducir el acoplamiento

La guía del encapsulamiento sugiere que sólo la información del *qué* puede hacer una clase debe ser visible desde el exterior, no *cómo* lo hace. Esto tiene una gran ventaja: si ninguna de las otras clases saben como nuestros datos son almacenados, entonces podemos cambiarlos fácilmente sin romper otras clases.

A diferencia de otros lenguajes, en Python no es posible negar el acceso a los campos de una clase. Es decir, una clase *podría* acceder a nuestros datos si lo intenta. Sin embargo, podemos separar el *qué* y el *cómo* a través de un *accesor*, definiendo así cómo debería comunicarse una clase externa con **Escena**.

```

1 class Escena:
2
3     #... Código omitido...
4     def getSalida(self, direccion):
5         if direccion == 'norte':
6             return self.salidaNorte
7         if direccion == 'este':
8             return self.salidaEste
9         if direccion == 'sur':
10            return self.salidaSur
11        if direccion == 'oeste':
12            return self.salidaOeste
13        return None

```

Una vez hecho este cambio en la clase **Escena**, también debemos cambiar la clase **Juego**. Cada vez que se accedía a una variable de salida, ahora debemos usar el método de acceso. Por ejemplo, en vez de escribir:

```
1 sgteEscena = self.escenaActual.salidaEste
```

Ahora debemos escribir:

```
1 sgteEscena = self.escenaActual.getSalida('este')
```

Esto hace que escribir una parte de la clase **Juego** sea más fácil también. El cambio sugerido acá resultará en el siguiente código para el método **irAEscena**:

```

1 sgteEscena = None
2 if direccion.equals('norte'):
3     sgteEscena = self.escenaActual.getSalida('norte')
4 if direccion.equals('este'):
5     sgteEscena = self.escenaActual.getSalida('este')
6 if direccion.equals('sur'):
7     sgteEscena = self.escenaActual.getSalida('sur')
8 if direccion.equals('oeste'):
9     sgteEscena = self.escenaActual.getSalida('oeste')

```

En vez de esto, podemos reducir todo este código en una sola línea:

```

1 sgteEscena = self.escenaActual.getSalida(direccion)

```

Hasta ahora no hemos cambiado la representación de las salidas de la clase `Escena`. Solo hemos limpiado la interfaz. El *cambio* en la clase `Juego` fue mínimo –en vez del acceso a una variable, utilizamos la llamada a un método– pero la ganancia es enorme. Ahora podemos cambiar la manera en que las salidas son almacenadas en una escena, sin la necesidad de preocuparse por romper algo en la clase `Juego`. La representación interna en `Escena` ha sido completamente desacoplada de la interfaz. Ahora que el diseño es como debió haber sido desde el principio, cambiar los campos de las salidas en la clase `Escena` por un diccionario es fácil.

```

1 class Escena:
2
3     def init (self, descripcion):
4         self.descripcion = descripcion
5         self.salidas = dict()
6
7     # Define las salidas de esta escena. Cada direccion
8     # lleva a otra escena o a None (cuando no hay salida)
9     def setSalidas(self, norte, este, sur, oeste):
10        if norte != None:
11            self.salidas['norte'] = norte
12        if este != None:
13            self.salidas['este'] = este
14        if sur != None:
15            self.salidas['sur'] = sur
16        if oeste != None:
17            self.salidas['oeste'] = oeste
18
19        # Retorna la escena a la que se llega si vamos desde
20        # esta escena en la direccion "direccion". Si no hay
21        # escena en esta direccion retornamos None
22        def getSalida(self, direccion):
23            if direccion in self.salidas:
24                return self.salidas[direccion]
25            else:
26                return None
27
28        # Retorna la descripcion de la escena (aquella que
29        # fue definida en el constructor)
30        def getDescription(self):

```

```
31     return self.descripcion
```

Es importante enfatizar nuevamente que podemos hacer este cambio sin siquiera comprobar si algo se romperá en otras partes de la aplicación. Puesto que sólo hemos cambiado aspectos privados de la clase `Escena`, las cuales por definición no pueden ser utilizadas en clases externas, este cambio no impacta en otras clases. La interfaz se mantiene sin cambios.

Una consecuencia de este cambio es que nuestra clase `Escena` es ahora incluso más corta. En vez de listar cuatros variables separadas, ahora tenemos sólo una. Además, el método `getSalida` fue simplificado de manera considerable.

Recordemos que el objetivo de esta serie de cambios era hacer más fácil la tarea de agregar dos posibles nuevas salidas: *arriba* y *abajo*. Esto ya se ha hecho mucho más fácil. Dado que ahora podemos usar el diccionario para almacenar las salidas de una escena, guardar estas dos nuevas direcciones funcionará sin hacer ningún cambio. También podemos obtener la información de la salida nueva a través del método `getSalida` sin ningún problema.

El único lugar en donde la información de la cantidad de salidas existentes (*norte*, *este*, *sur*, *oeste*) aún es parte del código es en el método `setSalidas`. Este es el último lugar que necesita ser mejorado. Por el momento, el contrato del método es de la siguiente manera:

```
1 def setSalidas(self, norte, este, sur, oeste)
```

Este método es parte de la interfaz de la clase `Escena`, por lo que cualquier cambio que hagamos afectará inevitablemente a otras clases debido al acoplamiento. Es importante notar que nunca podremos desacoplar por completo las clases de una aplicación; de otra manera objetos de clases diferentes nunca podrían interactuar entre sí. Más bien, lo que debemos intentar lograr es mantener el grado de acoplamiento lo más bajo posible. Si de todas maneras debemos hacer un cambio a `setSalidas` para acomodar una nueva dirección, nuestra solución preferida es reemplazarlo completamente por este método:

```
1 # Define la salida para esta escena
2 # direccion : la direccion de la salida
3 # vecina : la escena vecina en la direccion dada
4 def setSalidas(self, direccion, vecina):
5     self.salidas[direccion] = vecina
```

Ahora la salida de una escena puede ser asignada de manera individual por cada dirección, y cualquier dirección puede ser usada como una salida. En la clase `Juego`, el cambio que resulta de modificar la interfaz de `Escena` está dada de la siguiente manera. En vez de escribir:

```
1 lab.setSalidas(afuera, oficina, None, None)
```

Ahora escribimos:

```
1 lab.setSalida('norte', afuera)
2 lab.setSalida('este', oficina)
```

Con estos cambios hemos removido por completo la restricción que `Escena` puede contener sólo cuatro salidas. La clase `Escena` puede ahora guardar las salidas en las direcciones *arriba* y *abajo*, de igual manera que puede guardar cualquier otra dirección que se pueda pensar (noreste, sudeste, etc.).

17.5 Diseño basado en responsabilidades

Hemos visto en secciones anteriores que al usar apropiadamente el encapsulamiento podemos reducir el acoplamiento, y así podemos reducir significativamente la cantidad de trabajo necesaria para introducir cambios en una aplicación. Sin embargo, el encapsulamiento no es el único factor que influye en el grado de acoplamiento. Otro de estos aspectos es lo que se conoce con el nombre de *diseño basado en responsabilidades*.

El diseño basado en responsabilidades es el proceso de diseñar clases, asignando responsabilidades bien definidas a cada clase. Este proceso se puede usar para determinar cuál clase debe implementar qué parte de una función. En otras palabras, el diseño basado en responsabilidades expresa la idea que cada clase debe ser responsable de manejar sus propios datos. A menudo, cuando necesitamos agregar nuevas funcionalidades a una aplicación, lo que necesitamos es preguntarnos en qué clase debemos agregar un método para implementar esta nueva funcionalidad. ¿Cuál clase debe ser responsable de esta tarea? La respuesta es que la clase que es responsable de almacenar ciertos datos debe ser también responsable de manipularlos.

Hay que tener presente que la manera en que se usa el diseño basado en responsabilidades influye en el grado de acoplamiento, y así, en la facilidad con la que una aplicación se puede modificar o extender.

17.5.1 Responsabilidades y acoplamiento

Los cambios que introdujimos en la clase `Escena` hacen que ahora sea muy fácil agregar nuevas direcciones para los movimientos *arriba* y *abajo* en la clase `Juego`. Supongamos, por ejemplo, que queremos agregar una nueva escena (sótano) abajo de la oficina. Todo lo que tenemos que hacer es unos cambios menores en el método `crearEscena` de la clase para así crear la escena y dos salidas:

```
1 def crearEscena(self):
2     ...
3     sotano = Escena("en el sotano")
4     ...
5     oficina.setSalida("abajo", sotano)
6     sotano.setSalida("arriba", oficina)
```

Dada la nueva interfaz de la clase `Escena`, lo anterior funciona sin problemas. Así, los cambios resultan ahora muy fáciles de introducir, confirmando que el nuevo diseño es mejor.

Nuestro objetivo de reducir al máximo el acoplamiento implica que los cambios que hagamos en la clase `Escena` no incidan en tener que hacer cambios en la clase `Juego`.

Actualmente, aún tenemos en el código de la clase `Juego` la noción que la información que queremos de una escena consiste en un string de descripción y el string de salida:

```
1 print "Esta en " + escenaActual.getDescripcion()
2 print piezaActual.getStringSalida()
```

¿Qué pasaría si quisiéramos agregar objetos o tesoros en nuestro juego? Cuando describimos lo que vemos, la lista de tesoros, entre otros, toda esta información debe estar incluida en la descripción de la escena. Así, tendríamos que introducir no sólo cambios a la clase `Escena`, sino que también cambiar el código donde se imprime la descripción.

Este problema corresponde a una violación de la regla de diseño basado en responsabilidades. Dado que la clase `Escena` contiene la información de una escena, entonces también debiera ser capaz de dar una descripción de la misma. Esto lo podemos mejorar agregando el siguiente método a la clase:

```
1 # getDescripcionLarga: -> str
2 # Retorna la descripcion de una escena
3 def getDescripcionLarga(self):
4     return "Esta en " + self.descripcion + ". Salida hacia: " + \
5         self.getStringSalida()
```

Y así, en la clase `Juego` debiéramos escribir:

```
1 print self.escenaActual.getDescripcionLarga()
```

De esta manera, la *descripción larga* de una escena ahora incluye la descripción estándar y la información sobre las salidas. Además, es capaz de registrar en el futuro si se introducen nuevos cambios a la escena. Cuando hagamos estas extensiones, tendremos que modificar únicamente una clase: `Escena`.

17.6 Cohesión

Anteriormente en este capítulo introdujimos la idea de cohesión: una unidad de código debe ser responsable de una y sólo una tarea. A continuación veremos en más detalle el principio de cohesión y veremos ejemplos. Notemos que el principio de cohesión puede ser aplicado tanto a clases como a métodos.

17.6.1 Cohesión de métodos

Cuando hablamos de *cohesión de métodos*, buscamos expresar la idea que cada método debe ser responsable de una, y solamente una tarea bien definida.

Podemos ver un ejemplo de método cohesivo en la clase `Juego`. Esta clase tiene un método llamado `imprimirBienvenida` para mostrar el mensaje de bienvenida, y este método se llama cada vez que comienza el juego en el método `jugar`.

```
1 # jugar: ->
2 # Rutina principal: loop hasta el fin del juego
3 def jugar(self):
4     self.imprimirBienvenida()
5     terminado = False
6     while ! terminado:
7         comando = self.parser.getComando()
8         terminado = self.procesarComando(comando)
9     print "Gracias por jugar. Adios!"
10
11 # imprimirBienvenida: ->
12 # Imprime el mensaje de bienvenida para el jugador
13 def imprimirBienvenida(self):
14     print ""
15     print "Bienvenido!"
16     print "Escriba 'ayuda' si necesita ayuda."
```

```
17 print ""
18 print self.escenaActual.getDescripcionLarga()
```

Desde un punto de vista funcional, podríamos haber simplemente ingresado los comandos del método `imprimirBienvenida` directamente en el método `jugar` y así obtener exactamente el mismo resultado sin tener que definir un método adicional. Sin embargo, es mucho más fácil de entender y modificar el funcionamiento de un método si se utilizan definiciones cortas y cohesivas. Así, el contar con métodos razonablemente cortos, fáciles de entender, y con nombres que indican claramente el propósito, permiten mantener más fácilmente un programa.

17.6.2 Cohesión de clases

La regla de *cohesión de clases* indica que cada clase debe representar una única entidad bien definida en el dominio del problema.

Supongamos que queremos agregar nuevos elementos al juego. Por ejemplo, cada escena puede contener un elemento, y cada elemento tiene una descripción y un peso. El peso de un elemento puede ser usado más tarde para determinar si se puede recoger o no.

Una primera aproximación podría ser incluir dos campos a la clase `Escena`: `descripcionElemento` y `pesoElemento`. Esto podría funcionar. Luego, podríamos especificar los detalles de cada elemento en cada escena, y podríamos imprimirlos detalles cuando entremos a una escena en particular. Sin embargo, este enfoque no cuenta con un buen grado de cohesión: la clase `Escena` ahora describe una escena y un elemento. También sugiere que un elemento está unido a una escena en particular, lo que no necesariamente es cierto.

Un mejor diseño podría ser crear una clase aparte para los elementos, probablemente de nombre `Elemento`. Esta clase podría tener campos para una descripción y peso, y una escena simplemente tendría una referencia a tal objeto.

Los beneficios concretos de separar escenas y elementos en el diseño se puede ver si cambiamos un poco la especificación. En otra variante del juego, podríamos permitir no solamente tener un único elemento en cada escena, sino contar con un número indefinido de ellos. En el diseño usando una clase `Elemento` aparte, esto es fácil: basta con crear varios objetos y almacenarlos en una colección asociada a una escena. Por el contrario, con el primer enfoque de solución, este cambio sería casi imposible de realizar.

17.6.3 Cohesión para lograr legibilidad

Hay varias maneras en las que una alta cohesión mejora el diseño de software. Las dos más importantes son la *legibilidad* y la *reutilización*. En el ejemplo que vimos anteriormente, la cohesión del método `imprimirBienvenida` es claramente un ejemplo en el que al aumentar la cohesión logramos tener una clase más legible, y por ende, más fácil de entender y mantener.

17.6.4 Cohesión para lograr reutilización

La segunda gran ventaja de la cohesión es un alto potencial para la reutilización. Por citar un ejemplo, cuando creamos una nueva clase `Elemento` para manejar elementos en juego en las escenas, podemos crear múltiples de ellos y usar el mismo código para manejarlos.

La reutilización es también un aspecto importante de la cohesión de métodos. Consideremos un método de la clase `Escena` con la siguiente firma:

```

1 # dejarEscena: str -> Escena
2 def dejarEscena(self, direccion):
3     ...

```

Este método podría retornar la escena en una `direccion` dada, de tal manera que pueda ser usada como `self.escenaActual`, y además imprimir la descripción de la nueva escena a la cual el personaje del juego entra. Esta alternativa de diseño es posible, y puede efectivamente funcionar. Sin embargo, nosotros separamos esta tarea en dos métodos:

```

1 # getSalida: str -> Escena
2 def getSalida(self, direccion):
3     ...
4
5 # getDescripcionLarga: -> str
6 def getDescripcionLarga(self):
7     ...

```

El primer método es responsable de retornar la escena siguiente, mientras que el segundo genera un string con la descripción de la escena. La ventaja de este diseño es que las tareas así separadas pueden ser reutilizadas más fácilmente. Por ejemplo, el método `getDescripcionLarga` ahora se utiliza no sólo en el método `irAEscena`, sino que también en `imprimirBienvenida`. Esto es posible porque el método tiene un gran grado de cohesión. En efecto, no habríamos podido reutilizarlo en la versión con el método `dejarEscena`.

17.7 Refactoring

Cuando diseñamos aplicaciones, deberíamos intentar mirar más adelante, anticipar posibles cambios, y crear clases y métodos altamente cohesivos y no fuertemente unidos, de tal manera que podamos realizar modificaciones fácilmente. Si bien es una meta interesante, no siempre podemos anticiparnos a *todas* las posibles adaptaciones futuras, y luego no es factible desarrollar todas las posibles extensiones que quisiéramos. Es por esto que el *refactoring* es importante.

Refactoring (o *refactorización*) es la actividad de reestructurar clases y métodos existentes para adaptarlos a cambios en funcionalidad y requisitos. A menudo en el ciclo de vida de una aplicación, ciertas capas de funcionalidad se van agregando. Un efecto común de esto es que el tamaño de clases y métodos crece lentamente.

Es fácil caer en la tentación de agregar más código a las clases y métodos existentes. Sin embargo, el hacer esto, a menudo, disminuye el grado de cohesión. Cuando se agrega más y más código a una clase o método, es altamente posible que en cierto punto lleguen a representar más de una tarea o entidad claramente definida.

Así, el refactoring consiste en re-pensar y re-diseñar la estructura de clases y métodos. Más comúnmente, el efecto logrado es que las clases se dividen en dos, o que los métodos se dividen en dos o más. El refactoring también puede involucrar (en menor escala) la unión de múltiples clases en una sola, o de varios métodos en uno solo.

17.7.1 Refactoring y testing

Es importante que notemos que cada vez que realizamos refactoring en un programa, usualmente estamos proponiendo realizar cambios potencialmente grandes a algo que ya estaba funcionando correctamente. Como bien ya sabemos, cada vez que modificamos algo que funciona bien, es altamente

posible que se introduzcan errores de manera involuntaria. Luego, resulta crucial proceder con cautela, y antes de refactorizar un bloque de código, *asegurarnos que exista un conjunto de tests para el código actual*.

Una vez que dispongamos de un conjunto de tests, podemos comenzar el proceso de refactoring. Idealmente, involucra dos etapas:

1. El primer paso es refactorizar la estructura interna del código, sin introducir *ningún* cambio a la funcionalidad de la aplicación. En otras palabras, el nuevo programa debe comportarse en tiempo de ejecución exactamente igual que en su versión original. Una vez que hayamos completado esta etapa, debemos probar el nuevo código con el conjunto de tests que disponíamos originalmente, y asegurarnos que pasen todos, sin excepción.
2. La segunda etapa se realiza una vez que hayamos reestablecido la funcionalidad original en el código refactorizado. De esta manera, estamos en una posición segura para poder mejorar la expresividad del código del programa. Una vez que hayamos realizado esta etapa, claramente debemos volver a probar la funcionalidad con un conjunto de tests.

Capítulo 18

Interfaces y Polimorfismo

18.1 ¿Qué es una interfaz?

El concepto de interfaz es un concepto muy importante en la organización de programas. Veremos que el concepto de interfaz toma una dimensión adicional en el caso de la programación orientada al objeto, dado la vinculación con el concepto de polimorfismo, por lo que estudiaremos estos dos conceptos simultáneamente. Por el momento, nos enfocaremos en el concepto de interfaz.

Una interfaz es una especificación de como usar un módulo, sin especificar su implementación. Esto está basado en un principio fundamental de cómo organizar programas, que es el principio de separar interfaces de sus implementaciones. Por ejemplo, el módulo `math` de Python tiene una interfaz bien definida, en términos del conjunto de funciones (`math.sqrt(x)`, `math.cos(x)`, etc ...) y de constantes (`math.pi`, ...) que provee. Sin embargo, la documentación del módulo (disponible en <http://docs.python.org/2/library/math.html>), no dice nada sobre cómo esta implementada la función raíz cuadrada, o la función logaritmo, o ninguna de las otras funciones. Hay varias maneras de calcular la raíz cuadrada de un número, pero la interfaz no nos dice cuál en particular se usa. Esto es importante, porque al conocer detalles innecesarios sobre la implementación de un módulo uno puede “vincularse” con este módulo de manera muy fuerte. Como resultado, un cambio en la implementación del módulo puede repercutir en que los usuarios de este módulo tengan que modificar sus programas para adaptarse a dicho cambio.

Los usuarios de un módulo se llaman sus clientes, mientras que el módulo mismo corresponde al proveedor de funcionalidad basado en esta interfaz. En el caso de la programación orientada al objeto, la interfaz de un objeto es el conjunto de métodos que provee, es decir, el conjunto de métodos que uno puede ejecutar sobre el objeto.

18.2 Ejemplo: animales

Si estamos implementando una simulación de una granja, necesitamos una variedad de animales: cabras, vacas, cerdos, gallos, patos, caballos, conejos, etc. Además, cada tipo de animal tiene un comportamiento distinto: cada animal tiene un grito, tiene o no plumas, un número distinto de patas, etc. Implementando un programa orientado al objeto, podemos definir una interfaz general de nuestros animales. Por ejemplo, la siguiente definición de interfaz cubre varios casos:

```
1 # un animal de granja es un objeto que tiene las siguientes operaciones:
2
3 # grito: -> string
```

```

4  # devuelve una representacion textual del sonido que hace un animal
5
6  # plumas: -> bool
7  # devuelve True si el animal tiene plumas
8
9  # pelo: -> bool
10 # devuelve True si el animal tiene pelo
11
12 # patas: -> int
13 # devuelve el numero de patas del animal
14
15 # cola: -> bool
16 # devuelve si el animal tiene una cola o no
17
18 # nombre: -> string
19 # devuelve el nombre de la especie de animal
20
21 # comida: -> string
22 # devuelve el nombre de la comida favorita del animal

```

18.3 ¿Qué es el polimorfismo?

El polimorfismo es una propiedad de la programación orientada al objeto que permite interactuar con objetos con representación distinta de manera uniforme. Es decir, si varios objetos tienen una implementación distintas de la misma interfaz, el cliente debería ser capaz de procesarlos de la misma manera, sin saber que sus implementaciones varían.

Hasta ahora, si queremos procesar los animales de la granja, por ejemplo contar las patas en una lista de animales, podemos hacer algo similar a lo siguiente:

```

1  # contar_patas: lista(animales) -> int
2  # cuenta el numero de patas de los animales en la lista
3
4  def contar_patas(animales):
5      patas = 0
6      for animal in animales:
7          if animal.nombre == 'cerdo':
8              patas += 4
9          elif animal.nombre == 'pato':
10             patas += 2
11             ...
12             else: # ultimo caso del conejo
13                 patas += 4
14     return patas

```

La programación orientada al objeto nos hace ver el problema de una manera distinta. Como cada objeto es dueño de su comportamiento, cada animal tiene que saber cuántas patas tiene. Es decir, un cerdo sabe que tiene cuatro patas, mientras que un pata sabe que tiene dos patas. Intuitivamente, queremos algo similar a lo siguiente:

```
1 class Cerdo:
```

APUNTE DE USO INTERNO

PROHIBIDA SU DISTRIBUCIÓN

```

2
3     # ...
4
5     # patas: -> int
6     def patas(self):
7         return 4
8
9 class Pato:
10
11     # ...
12
13     # patas: -> int
14     def patas(self):
15         return 2
16
17 p = Pato()
18 c = Cerdo()
19 assert p.patas() == 2
20 assert c.patas() == 4

```

En Python, este código funciona sin ningún modificación, y es uno de los ejemplos los mas basicos de polimorfismo: dos objetos pueden responder al mismo mensaje con una implementación distinta del método, en este caso del método `patas`. Dinámicamente (es decir, durante la ejecución del programa), Python determina cuál es la clase a la cual pertenece cada objeto, y ejecuta la implementación del método relevante para dicha clase. De hecho, cada clase definida en Python tiene una tabla de métodos, que contiene los métodos asociado a cada clase. Su estructura es similar a un diccionario asociando nombres de métodos a los métodos actuales (la verdadera implementación de esto es más compleja, por un tema de eficiencia).

18.4 Beneficios del polimorfismo

Valiéndose de este nuevo conocimiento, podemos reescribir el código de la función `contar_patas` usando polimorfismo. Su implementación va a ser mucho más simple:

```

1 # contar_patas: lista(animales) -> int
2 # cuenta el numero de patas de los animales en la lista
3
4 def contar_patas(animales):
5     patas = 0
6     for animal in animales:
7         patas += animal.patas()
8     return patas

```

En esencia, en vez de hacer una cadena de instrucciones `if/else` nosotros mismos, dejamos que el intérprete de Python se encargue de esto usando el polimorfismo: cada animal va a ejecutar la versión apropiada del método `patas` según su clase; Python se encarga de buscar el método.

La única restricción es que cada animal *tiene* que tener una implementación de `patas`. Si no, el programa va a producir un error de tipo `AttributeError`.

Esto tiene varios beneficios:

- El código de `contar_patas` se simplifica, y por lo tanto es más abstracto y más entendible (si uno no está interesado en la implementación de `patas`).
- Este código también es más fácilmente extensible. Si uno agrega un nuevo animal, no es necesario cambiar la implementación de `contar_patas`, basta con agregar la implementación correcta de `patas` para el nuevo tipo de animales.
- No tenemos que escribir la instrucción condicional, lo que nos permite evitar errores.
- Mientras más uso del método `patas` y más clases implementando `patas` hay en el programa, más importante son todos estos beneficios. Si hay diez usos de `patas`, ahorramos diez cadenas de instrucciones condicionales. Si hay cinco clases implementando `patas`, ahorramos diez cadenas de cinco instrucciones `if/else`, lo que es significativo. Además, las posibilidades de insertar errores en el código cuando uno agrega una nueva clase son mayores sin el uso del polimorfismo: es fácil cometer el error de cambiar nueve de las cadenas de instrucciones condicionales, y olvidar cambiar la última...

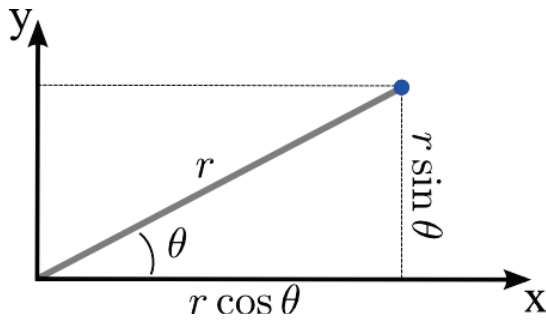
Sin embargo, el polimorfismo no es una solución perfecta. Hay varias consecuencias negativas que hay que tener en cuenta. En particular:

- Si uno está interesado en entender cómo funciona el comportamiento del método `patas`, uno tiene ahora que revisar múltiples ubicaciones en el código fuente, y no solamente una única ubicación como antes.
- Además, si uno quiere extender la interfaz de los animales (agregando el método `grito`, por ejemplo), es necesario modificar cada clase de animales, en vez de hacer el cambio en un único lugar.

18.5 Otro ejemplo: puntos y líneas

Los animales nos permitan ilustrar el polimorfismo de manera simple, pero un lector cuidadoso se habrá dado cuenta que en el caso de los animales el uso del polimorfismo no es tan necesario. Una manera eficiente de resolver el problema es agregar atributos a cada animal, y devolver dicho atributo. Una clase `Animal` general tendría un atributo `numeroDePatas`, un atributo `grito`, etc. Esto tendría beneficios parecidos al polimorfismo, salvo la desventaja que cada objeto tiene más variables, y ocupa más espacio en la memoria.

Sin embargo, en otros casos esta solución simple no es suficiente. Un ejemplo de esto es puntos en dos dimensiones. Hay dos maneras de representar coordenadas: como puntos cartesianos (con atributos `x` y `y`), o como puntos con notación polar (con atributos `angulo` y `distancia`). La figura siguiente nos muestra la diferencia:



El polimorfismo nos permite unificar estos dos tipos de puntos detrás de una misma interfaz. De esta forma, un usuario puede usar puntos cartesiano o polares de la misma forma, y es más fácil cambiar la implementación si es necesario. El código de ambas clases se encuentra a continuación:

```

1  Class Cartesian:
2      def __init__(self, x, y):
3          self.__x = x
4          self.__y = y
5
6      # x: -> num
7      def x(self):
8          return self.__x
9
10     # y: -> num
11     def y(self):
12         return self.__y
13
14     # radius: -> num
15     def radius(self):
16         return math.sqrt(self.x() ** 2 + self.y() ** 2)
17
18     # angulo: -> num
19     def angulo(self):
20         return math.atan2(self.y(), self.x())
21
22     # __add__: punto -> punto
23     # suma de puntos
24     def __add__(self, p2):
25         return Cartesian(self.x() + p2.x(), self.y() + p2.y())
26
27     # __sub__: punto -> punto
28     # resta de puntos
29     def __sub__(self, p2):
30         return Cartesian(self.x() - p2.x(), self.y() - p2.y())
31
32     # __str__: -> string
33     # convierte un punto en un string para imprimirlo en la pantalla
34     def __str__(self):
35         return "(x=" + str(self.x()) + ", y=" + str(self.y()) + ")"
36
37     class Polar:
38         def __init__(self, d, a):

```

```

39     self.__distancia = d
40     self.__angulo = a
41
42     # x: -> num
43     def x(self):
44         return self.radius() * math.cos(self.angulo())
45
46     # y: -> num
47     def y(self):
48         return self.radius() * math.sin(self.angulo())
49
50     # radius: -> num
51     def radius(self):
52         return self.__distancia
53
54     # angulo: -> num
55     def angulo(self):
56         return self.__angulo
57
58     # __add__: punto -> punto
59     # suma de puntos
60     def __add__(self, p2):
61         return Cartesian(self.x() + p2.x(), self.y() + p2.y())
62
63     # __sub__: punto -> punto
64     # resta de puntos
65     def __sub__(self, p2):
66         return Cartesian(self.x() - p2.x(), self.y() - p2.y())
67
68     # __str__: -> string
69     # convierte un punto un string para imprimirlo en la pantalla
70     def __str__(self):
71         return "(r=" + str(self.distancia()) + ", theta=" + str(self.angulo()) + ")"

```

Cada clase tiene la misma interfaz, pero los comportamientos en cada casos son muy distintos. Cuando una clase hace un acceso a un atributo, la otra hace un cálculo, pero las interfaces son iguales. Otra clase, la clase `Linea`, puede ocupar cualquier tipo de puntos como atributos, dado que se conforman a la misma interfaz:

```

1 class Linea:
2     def __init__(self, p1, p2):
3         self.__p1 = p1
4         self.__p2 = p2
5
6     def p1(self):
7         return self.__p1
8
9     def p2(self):
10         return self.__p2
11
12     def largo(self):
13         return distancia(self.p1().x(), self.p1().y(), self.p2().x(), self.p2().y())

```

Además, veamos dos funcionalidades adicionales de Python: la definición de los metodos `__add__` y otras, que permite a Python usar el operador `+` para los puntos. Hay varios operadores que se pueden definir en forma similar. Esto también es una aplicación del concepto de polimorfismo: todas las clases que tienen el método `__add__` se invocan con el operador `+`, todas las que tienen `__str__` se van a imprimir en pantalla como queremos al usar la función `str()`, etc.

```
1 >>> from puntos import Cartesian
2 >>> a = Cartesian(1,1)
3 >>> b = Cartesian(2,2)
4 >>> c = a + b
5 >>> c
6 (x=3, y=3)
```

La segunda funcionalidad es nombrar atributos como `__x`, lo que permite Python forzar la encapsulación. Tratar de acceder a un atributo con guiones bajos afuera de la clase resulta en una error:

```
1 >>> from puntos import Cartesian
2 >>> a = Cartesian(1,1)
3 >>> a.__x
4 Traceback (most recent call last):
5   File "<stdin>", line 1, in <module>
6 AttributeError: Cartesian instance has no attribute '__x'
7 >>>
```

18.6 Reuso de código con delegación y herencia

Si miramos más de cerca el código de las clases para puntos cartesianos y polares, veremos que hay duplicación de código: en ambos casos, los metodos de suma y de resta son iguales. En este ejemplo particular esto no es un problema grande, pero existe la posibilidad que dos clases que tienen la misma interfaz tienen que compartir mucho código, lo que podría resultar en mucho código duplicado. Hay dos estrategias para resolver el problema:

- Crear una clase a parte que contiene el comportamiento común, y delegar el este comportamiento a esta clase.
- Compartir el comportamiento común con herencia.

18.6.1 Delegación

La primera posibilidad es crear una clase que maneje las operaciones de puntos. Por ejemplo, se podría llamar `PointOperator`, y esto es una posible implementación del método de suma. Métodos similares se pueden hacer para implementar el comportamiento compartido.

```
1 class PointOperator:
2
3     # constructor: punto punto -> PointOperator
4     def __init__(self, p1, p2):
5         self.p1 = p1
6         self.p2 = p2
```

```
7
8 # add: -> punto
9 def add(self):
10     return Cartesian(self.p1.x() + self.p2.x(), self.p1.y() + self.p2.y())
11
12 # ...
```

En seguida, podemos remplazar el código duplicado por un uso de la funcionalidad de `PointOperator`:

```
1 class Cartesian:
2     # ...
3
4     def __add__(self, p2):
5         return PointOperator(p1, p2).add()
6
7 class Polar:
8     # ...
9
10    def __add__(self, p2):
11        return PointOperator(p1, p2).add()
```

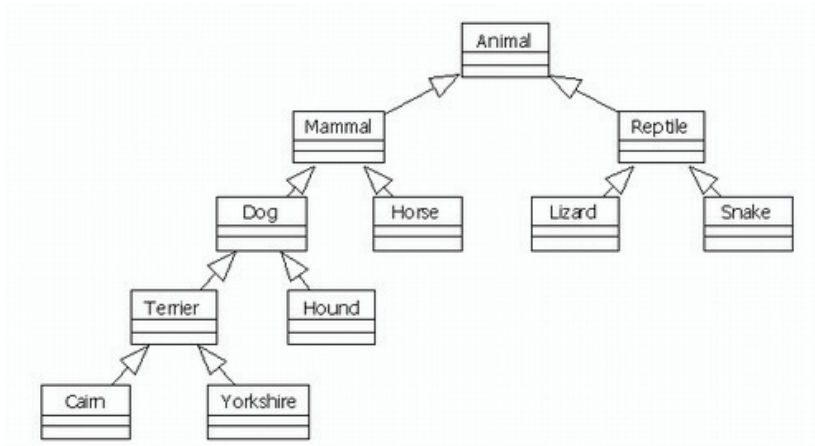
Una estrategia siempre exitosa es dividir las clases en entidades más pequeñas que se comunican. La delegación hace uso de este principio: el comportamiento común se traslada a una nueva clase, y ambas clases hacen uso de esta clase cuando necesitan el comportamiento común. Esta estrategia permite reutilizar el código duplicado a bajo costo. Notar que todavía queda un poco de duplicación de código, pero mucho menos que en la implementación original.

18.6.2 Herencia

La otra alternativa es definir una clase más general que contiene el comportamiento común, y ver las clases de puntos cartesianos y polares como casos específicos de esta clase general. Muchos lenguajes orientados al objeto (incluyendo Python) permiten hacer esto. En este caso, sería natural tener una clase general de puntos, y casos específicos de puntos cartesianos y polares.

El mecanismo de *herencia* nos permite hacer eso. La clase para punto va a ser llamada la superclase, y las clases de puntos cartesiano y polares van a ser las subclases, que hereden de la clase punto. El conjunto de clases forma una jerarquía de clases, con las clases las más generales arriba y las clases las más específicas abajo.

Otro ejemplo de herencia es hacer una jerarquía de animales, como la siguiente:



Notar que la herencia permite describir fácilmente estructuras de tipo árbol, es decir, cada clase tiene una superclase. Tener más de una superclase a la vez es algo mucho más complejo. En dichos casos, es mejor usar la composición y delegación.

Concretamente, ¿qué nos permite hacer la herencia? Nos permite reutilizar código de manera sencilla: una subclase hereda todos los datos y el comportamiento definido en la superclase. Si una clase no define para sí misma un método, se busca este método en la superclase, si lo tiene. De esta manera, podemos definir los métodos de suma y resta en la clase punto, y agregar en esta clase cualquier otro comportamiento compartido (por ejemplo, si los puntos tienen un color, este aspecto es independiente del hecho que sean cartesianos o polares). El código Python ocupando herencia es el siguiente:

```

1 class Point:
2
3     def __add__(p2):
4         return Cartesian(self.p1.x() + self.p2.x(), self.p1.y() + self.p2.y())
5
6 class Cartesian(Point):
7     # no define __add__
8
9 class Polar(Point):
10    # no define __add__

```

En caso que sea necesario, se puede llamar al constructor de la superclase, o cualquier otro método, de la siguiente manera:

```

1 class Point:
2     def __init__(self):
3         # código importante
4
5 class Cartesian(Point):
6     def __init__(self, x, y):
7         Point.__init__(self)
8         self.x = x
9         self.y = y

```

Finalmente, uno tiene que ser precavido al usar herencia. Esto es por varias razones. La primera es que la herencia es un mecanismo más complejo que la delegación, por los posibles problemas de

superclase, etc. La segunda es que es sólo es posible heredar de una superclase, mientras que una clase puede delegar a varias clases. Finalmente, y lo más importante, la herencia tiene un sentido muy específico.

Consideremos el ejemplo de las ruedas de una bicicleta. Uno podría pensar que una bicicleta herede de una rueda, para reutilizar su compartamiento, pero esto sería muy incorrecto. Para saber si una clase A hereda de otra clase B, hay que preguntarse: ¿Será un objeto de tipo A también un objeto de tipo B? En este ejemplo, ¿será una bicicleta una rueda? Absolutamente no. Una bicicleta *tiene* una rueda, *no es* una rueda. Esto nos muestra que la herencia en este caso es el mecanismo equivocado. Lo que en verdad queremos es la composición de objetos: una bicicleta contiene un atributo de tipo rueda. De hecho, la bicicleta tendría dos atributos de tipo rueda, algo que no podemos hacer con herencia.